

# Guia de Ejemplos por Estilo Arquitectonico

Conexion entre Estilos Arquitectonicos,  
Patrones Arquitectonicos y Patrones de Diseno

*Guia tecnica para el desarrollo del taller  
Arquitectura de Software*

## Introduccion

Esta guia presenta un ejemplo completo para cada uno de los cinco estilos arquitectonicos principales, mostrando como se conectan los tres niveles de abstraccion en el diseno de software:

- **Estilo arquitectonico:** Modelo general de organizacion del sistema (alto nivel).
- **Patron arquitectonico:** Solucion estructural dentro del estilo elegido (nivel medio).
- **Patron de diseno:** Solucion detallada en el codigo fuente (bajo nivel).

Cada ejemplo sigue la misma estructura para que el estudiante pueda identificar claramente cada nivel y replicar el analisis en su propio taller.

## Estructura de cada ejemplo

Cada estilo arquitectonico se desarrolla en 5 secciones:

- Descripcion del caso de uso (nombre, objetivo, funciones clave)
- Seleccion y justificacion del estilo arquitectonico
- Aplicacion de patrones arquitectonicos (tabla detallada)
- Aplicacion de patrones de diseno (tabla detallada)
- Diseno basico de la arquitectura

## Tabla resumen de niveles

| Nivel                 | Que es?                                | Ejemplos                                                          |
|-----------------------|----------------------------------------|-------------------------------------------------------------------|
| Estilo arquitectonico | Modelo general de organizacion         | Microservicios, Cliente-Servidor, Capas, Event-Driven, Serverless |
| Patron arquitectonico | Solucion estructural dentro del estilo | MVC, API Gateway, Repository, CQRS, Service Discovery             |
| Patron de diseno      | Solucion detallada en el codigo        | Singleton, Factory, Observer, Strategy, Adapter, Builder          |

## EJEMPLO 1: Estilo Microservicios

### 1. Descripcion del caso de uso

**Nombre de la aplicacion:** MiReservaPro

**Objetivo principal:** Permitir a los usuarios reservar servicios en linea (peluquerias, talleres, asesorias) con alta disponibilidad y escalabilidad.

**Funciones clave:**

- Registro y autenticacion de usuarios
- Busqueda de servicios por ubicacion y categoria
- Reserva de citas
- Pagos en linea
- Notificaciones por correo electronico
- Panel de administracion para prestadores
- Escalabilidad para multiples ciudades y paises

### 2. Seleccion del estilo arquitectonico

**Estilo elegido:** Microservicios

**Justificacion:**

- Permite dividir la aplicacion en modulos independientes (usuarios, reservas, pagos, notificaciones).
- Cada servicio puede escalarse de forma autonoma segun la demanda.
- Facilita el despliegue continuo y la resiliencia ante fallos.
- Ideal para aplicaciones que crecieran en funcionalidad y volumen de usuarios.

### 3. Aplicacion de patrones arquitectonicos

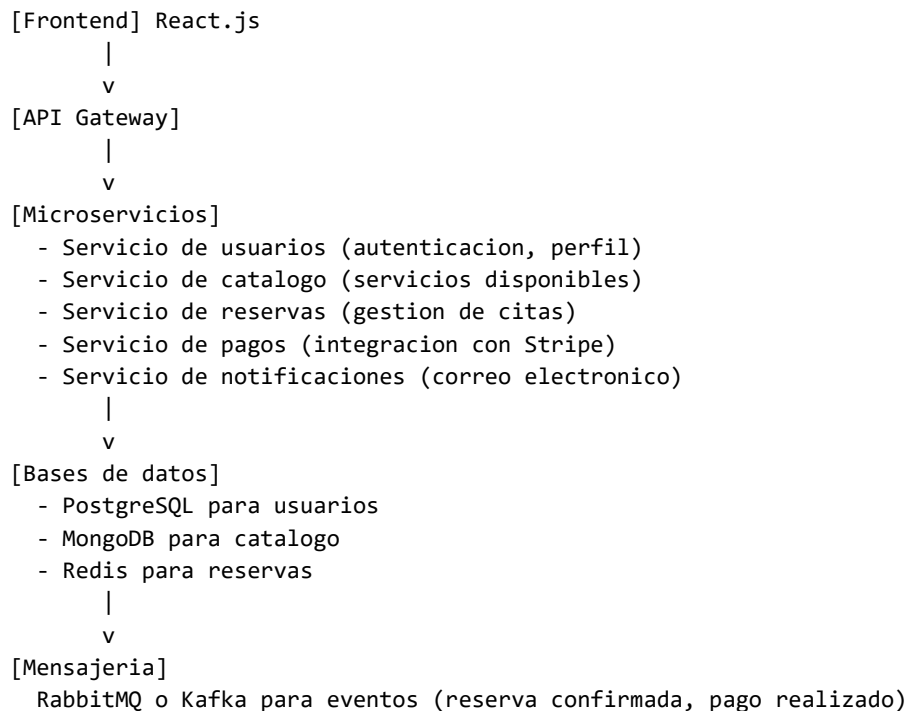
| Patron arquitectonico | Que problema resuelve?                                             | Donde se aplica en la arquitectura? |
|-----------------------|--------------------------------------------------------------------|-------------------------------------|
| API Gateway           | Centraliza el acceso a los microservicios y gestiona autenticacion | Punto de entrada para el frontend   |
| Service Discovery     | Permite que los servicios se encuentren entre si dinamicamente     | Backend, entre microservicios       |
| Circuit Breaker       | Evita que fallos en un servicio afecten a todo el sistema          | En servicios criticos como pagos    |
| Database per Service  | Aisla los datos por servicio,                                      | Cada microservicio tiene su         |

|                        |                                        |                                                 |
|------------------------|----------------------------------------|-------------------------------------------------|
|                        | evitando acoplamiento                  | propia base de datos                            |
| Event-Driven (Pub/Sub) | Comunicacion asincrona entre servicios | Para notificaciones y actualizaciones de estado |

#### 4. Aplicacion de patrones de diseno

| Patron de diseno | Que problema resuelve?                                     | Donde se aplica?                                                    |
|------------------|------------------------------------------------------------|---------------------------------------------------------------------|
| Factory          | Crear objetos de reserva sin exponer la logica de creacion | Servicio de reservas: crear diferentes tipos de citas               |
| Singleton        | Garantizar una unica instancia de configuracion            | Conexion a base de datos y configuracion de cada microservicio      |
| Observer         | Notificar a multiples componentes cuando ocurre un evento  | Servicio de notificaciones: enviar email cuando se confirma reserva |

#### 5. Diseno basico de la arquitectura



#### Por que esta combinacion es adecuada?

- **Seguridad:** El API Gateway permite aplicar politicas de autenticacion y autorizacion centralizadas.
- **Escalabilidad:** Cada servicio puede escalarse segun su carga.
- **Resiliencia:** El patron Circuit Breaker protege el sistema ante fallos en servicios externos.
- **Mantenibilidad:** Los servicios estan desacoplados, lo que facilita actualizaciones y pruebas.

- **Desempeno:** La comunicacion asincrona con eventos mejora la respuesta del sistema.

## EJEMPLO 2: Estilo Cliente-Servidor

### 1. Descripcion del caso de uso

**Nombre de la aplicacion:** GestorDocumental

**Objetivo principal:** Aplicacion web para la gestion de documentos corporativos donde los empleados suben, consultan y descargan archivos desde un servidor centralizado.

**Funciones clave:**

- Autenticacion de usuarios con roles (admin, editor, lector)
- Subida y descarga de documentos
- Organizacion por carpetas y etiquetas
- Busqueda de documentos por nombre, fecha o etiqueta
- Control de versiones de archivos
- Registro de auditoria de accesos

### 2. Seleccion del estilo arquitectonico

**Estilo elegido:** Cliente-Servidor

**Justificacion:**

- La logica de negocio y el almacenamiento se centralizan en el servidor, facilitando el control de acceso.
- Los clientes (navegadores web) solo se encargan de la presentacion e interaccion.
- Ideal para aplicaciones empresariales donde el servidor gestiona datos sensibles de forma centralizada.
- Simplifica la administracion al tener un unico punto de despliegue del backend.

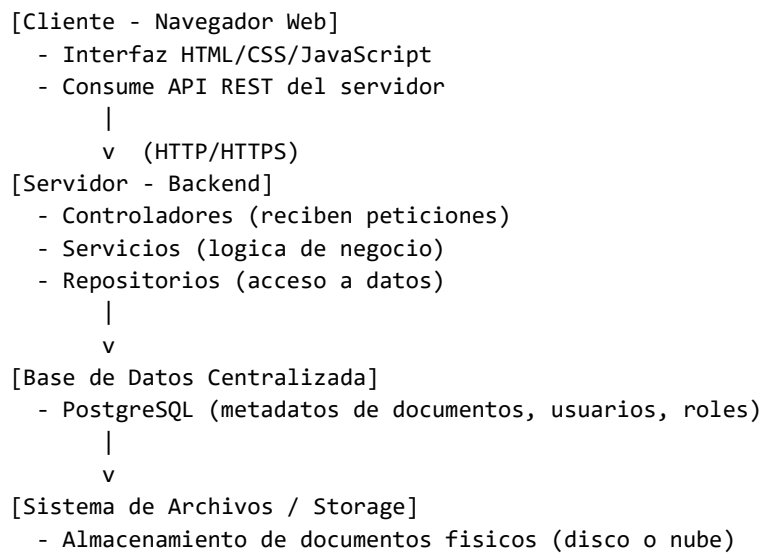
### 3. Aplicacion de patrones arquitectonicos

| Patron arquitectonico       | Que problema resuelve?                                                         | Donde se aplica en la arquitectura?                          |
|-----------------------------|--------------------------------------------------------------------------------|--------------------------------------------------------------|
| MVC (Model-View-Controller) | Separa la logica de negocio, la presentacion y el control de flujo             | Estructura interna del servidor web (ej: Django, Spring MVC) |
| Repository                  | Abstrae el acceso a datos desacoplando la logica de negocio de la persistencia | Capa de acceso a datos del servidor                          |
| DTO (Data Transfer Object)  | Transfiere datos entre cliente y servidor sin exponer el modelo interno        | API REST entre frontend y backend                            |

#### 4. Aplicacion de patrones de disenio

| Patron de disenio | Que problema resuelve?                                                | Donde se aplica?                                            |
|-------------------|-----------------------------------------------------------------------|-------------------------------------------------------------|
| Singleton         | Garantizar una unica instancia del pool de conexiones a BD            | Servidor: gestion de conexiones a la base de datos          |
| Strategy          | Permitir diferentes algoritmos de busqueda intercambiables            | Busqueda de documentos: por nombre, por fecha, por etiqueta |
| Adapter           | Integrar diferentes sistemas de almacenamiento con una interfaz comun | Almacenamiento: disco local, Amazon S3 o Azure Blob Storage |

#### 5. Diseno basico de la arquitectura



#### Por que esta combinacion es adecuada?

- **Control centralizado:** Toda la logica y los datos residen en el servidor, facilitando la seguridad y las politicas de acceso.
- **Separacion de responsabilidades:** MVC organiza el codigo del servidor de forma clara y mantenible.
- **Flexibilidad:** El patron Strategy permite cambiar algoritmos de busqueda sin modificar el resto del sistema.
- **Portabilidad:** El patron Adapter permite migrar el almacenamiento sin afectar la logica de negocio.

## EJEMPLO 3: Estilo Arquitectura en Capas

### 1. Descripción del caso de uso

**Nombre de la aplicación:** AcademiaOnline

**Objetivo principal:** Plataforma educativa web donde estudiantes se inscriben en cursos, visualizan contenido multimedia y realizan evaluaciones en línea.

**Funciones clave:**

- Registro e inicio de sesión de estudiantes y profesores
- Catálogo de cursos con descripción y temario
- Reproducción de videos y descarga de materiales
- Evaluaciones con calificación automática
- Panel del estudiante con progreso y certificados
- Panel del profesor para gestión de contenidos

### 2. Selección del estilo arquitectónico

**Estilo elegido:** Arquitectura en Capas (Layered Architecture)

**Justificación:**

- Organiza el sistema en capas horizontales con responsabilidades bien definidas.
- Cada capa solo se comunica con la capa inmediatamente inferior, reduciendo el acoplamiento.
- Facilita el mantenimiento y las pruebas unitarias por capa.
- Ideal para aplicaciones con lógica de negocio bien estructurada y predecible.

### 3. Aplicación de patrones arquitectónicos

| Patrón arquitectónico       | Que problema resuelve?                                           | Donde se aplica en la arquitectura?                                    |
|-----------------------------|------------------------------------------------------------------|------------------------------------------------------------------------|
| MVC (Model-View-Controller) | Separa la presentación de la lógica de negocio y los datos       | Capa de presentación: controladores y vistas                           |
| Repository                  | Abstrae las consultas a la base de datos con una interfaz limpia | Capa de acceso a datos: repositorios de cursos, usuarios, evaluaciones |
| Service Layer               | Encapsula la lógica de negocio en servicios reutilizables        | Capa de negocio: servicios de inscripción, calificación, certificación |

## 4. Aplicacion de patrones de disenio

| Patron de disenio | Que problema resuelve?                                                           | Donde se aplica?                                                                       |
|-------------------|----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| Factory Method    | Crear diferentes tipos de evaluacion (opcion multiple, verdadero/falso, abierta) | Servicio de evaluaciones: genera el tipo de examen segun la configuracion del profesor |
| Observer          | Notificar al estudiante cuando hay nuevos contenidos o resultados                | Sistema de notificaciones: suscripcion a eventos de cursos                             |
| Builder           | Construir objetos complejos paso a paso (certificados con multiples campos)      | Generacion de certificados: nombre, curso, fecha, calificacion, firma                  |

## 5. Diseno basico de la arquitectura

```
=====
CAPA DE PRESENTACION
- Interfaz web (HTML/CSS/JavaScript)
- Controladores (reciben solicitudes HTTP)
=====
|
v
=====
CAPA DE LOGICA DE NEGOCIO
- Servicio de inscripcion
- Servicio de evaluaciones
- Servicio de certificacion
- Servicio de contenidos
=====
|
v
=====
CAPA DE ACCESO A DATOS
- Repositorio de usuarios
- Repositorio de cursos
- Repositorio de evaluaciones
=====
|
v
=====
CAPA DE PERSISTENCIA
- Base de datos MySQL
- Almacenamiento de archivos multimedia
=====
```

### Por que esta combinacion es adecuada?

- **Organizacion clara:** Cada capa tiene una responsabilidad unica, lo que facilita entender y mantener el sistema.

- **Testabilidad:** Se puede probar cada capa de forma independiente con mocks de la capa inferior.
- **Reutilizacion:** Los servicios de la capa de negocio pueden ser consumidos por diferentes interfaces (web, movil).
- **Flexibilidad en datos:** El patron Repository permite cambiar la base de datos sin afectar las capas superiores.

## EJEMPLO 4: Estilo Event-Driven (Orientado a Eventos)

### 1. Descripción del caso de uso

**Nombre de la aplicación:** LogiTrack

**Objetivo principal:** Sistema de seguimiento de envíos en tiempo real para una empresa de logística, donde cada cambio de estado de un paquete genera eventos que actualizan dashboards, notifican a clientes y alimentan reportes.

**Funciones clave:**

- Registro de envíos con origen, destino y contenido
- Seguimiento en tiempo real del estado del paquete
- Notificaciones push y por email al cliente en cada cambio de estado
- Dashboard operativo para el centro de distribución
- Generación de reportes de desempeño logístico
- Integración con dispositivos IoT de los vehículos de transporte

### 2. Selección del estilo arquitectónico

**Estilo elegido:** Event-Driven Architecture

**Justificación:**

- El negocio gira en torno a eventos (paquete recogido, en tránsito, entregado) que deben propagarse en tiempo real.
- Los componentes reaccionan a eventos de forma desacoplada, sin necesidad de comunicación directa.
- Permite procesar grandes volúmenes de eventos concurrentes provenientes de múltiples vehículos y bodegas.
- Ideal para sistemas donde la reactividad y el procesamiento asíncrono son críticos.

### 3. Aplicación de patrones arquitectónicos

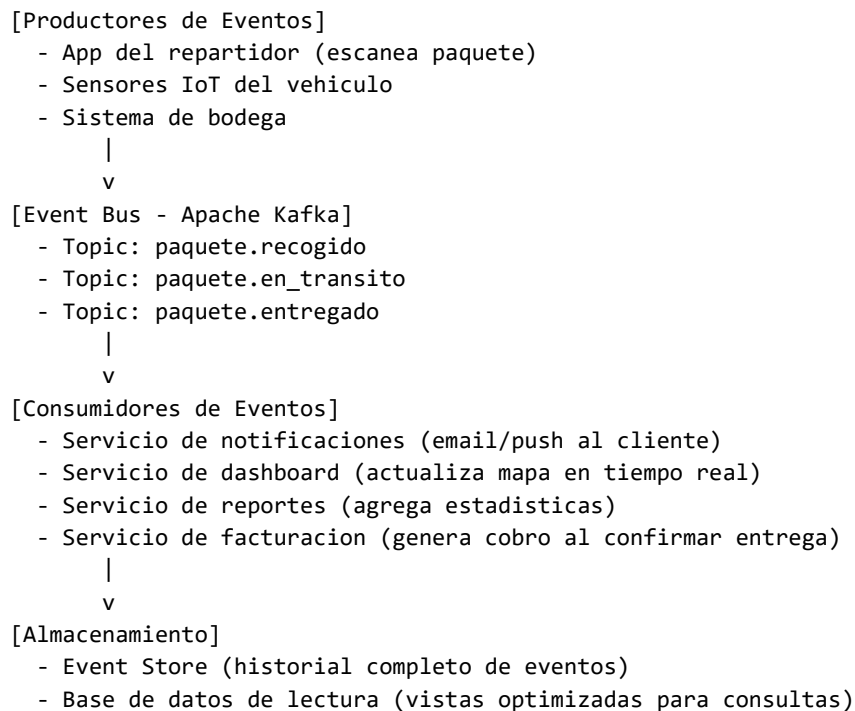
| Patrón arquitectónico                           | Que problema resuelve?                                                           | Donde se aplica en la arquitectura?                                             |
|-------------------------------------------------|----------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Event Bus / Message Broker                      | Canal central para publicar y suscribirse a eventos                              | Kafka o RabbitMQ como bus de eventos entre todos los componentes                |
| CQRS (Command Query Responsibility Segregation) | Separa las operaciones de escritura de las de lectura para optimizar rendimiento | Escritura: registrar eventos de estado; Lectura: consultar dashboard y reportes |
| Event Sourcing                                  | Almacenar el historial completo                                                  | Historial de estados del                                                        |

|  |                                  |                                             |
|--|----------------------------------|---------------------------------------------|
|  | de eventos como fuente de verdad | paquete: cada cambio es un evento inmutable |
|--|----------------------------------|---------------------------------------------|

#### 4. Aplicacion de patrones de disenio

| Patron de disenio | Que problema resuelve?                                               | Donde se aplica?                                                                                |
|-------------------|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Observer          | Notificar a multiples suscriptores cuando se produce un evento       | Cuando un paquete cambia de estado: notificar al cliente, al dashboard y al sistema de reportes |
| Strategy          | Seleccionar dinamicamente el canal de notificacion                   | Notificaciones: elegir entre email, SMS o push segun preferencia del cliente                    |
| Singleton         | Garantizar una unica instancia de la conexcion al broker de mensajes | Conexion a Kafka: evitar multiples conexiones innecesarias                                      |

#### 5. Diseno basico de la arquitectura



#### Por que esta combinacion es adecuada?

- **Tiempo real:** Los eventos se propagan instantaneamente a todos los consumidores interesados.
- **Desacoplamiento:** Los productores no necesitan conocer a los consumidores; solo publican eventos al bus.

- **Escalabilidad:** CQRS permite escalar lecturas y escrituras de forma independiente.
- **Auditabilidad:** Event Sourcing conserva el historial completo, permitiendo reconstruir el estado en cualquier momento.

## EJEMPLO 5: Estilo Serverless

### 1. Descripcion del caso de uso

**Nombre de la aplicacion:** FotoAlbumIA

**Objetivo principal:** Aplicacion web donde los usuarios suben fotografias que son analizadas automaticamente por inteligencia artificial para clasificarlas por contenido (paisajes, personas, comida, etc.) y generar albumes inteligentes.

**Funciones clave:**

- Registro de usuarios con autenticacion social (Google, Facebook)
- Subida de fotos desde navegador o movil
- Clasificacion automatica con IA (reconocimiento de imagenes)
- Creacion automatica de albumes por categoria
- Compartir albumes con enlaces publicos o privados
- Redimensionado automatico de imagenes (thumbnails)

### 2. Seleccion del estilo arquitectonico

**Estilo elegido:** Serverless

**Justificacion:**

- No se requiere administrar servidores; las funciones se ejecutan bajo demanda.
- El costo es proporcional al uso real (pago por invocacion), ideal para trafico variable.
- Cada funcion es independiente y se escala automaticamente.
- Ideal para flujos de trabajo basados en disparadores (un archivo se sube y desencadena el procesamiento).

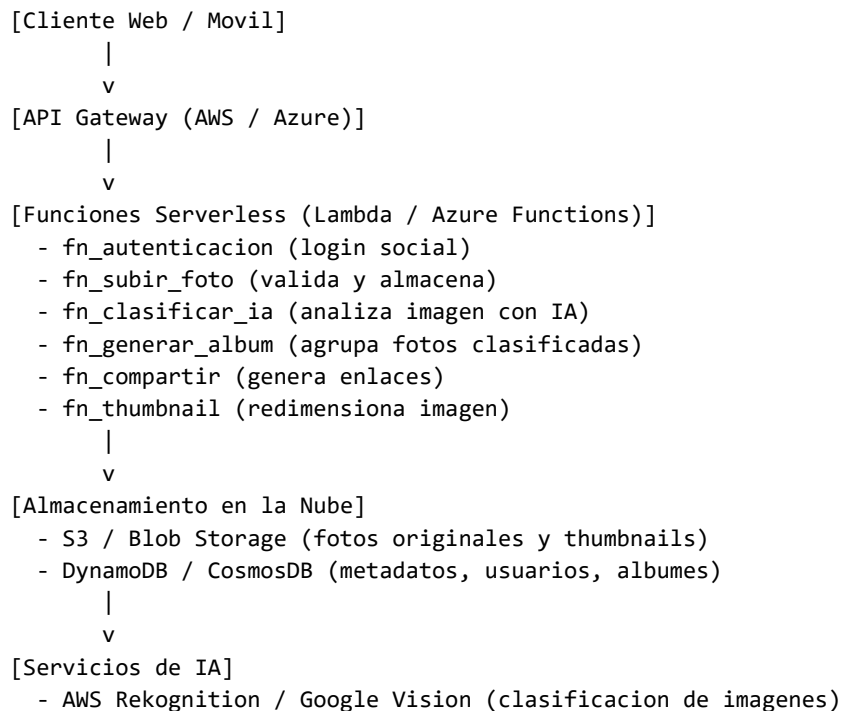
### 3. Aplicacion de patrones arquitectonicos

| Patron arquitectonico      | Que problema resuelve?                                                 | Donde se aplica en la arquitectura?                                  |
|----------------------------|------------------------------------------------------------------------|----------------------------------------------------------------------|
| API Gateway                | Punto de entrada unico que enruta peticiones a las funciones correctas | AWS API Gateway o Azure API Management frente a las funciones Lambda |
| Backend for Frontend (BFF) | Adapta las respuestas del backend para cada tipo de cliente            | Una funcion Lambda para la web y otra optimizada para la app movil   |
| Event-Driven Triggers      | Ejecutar funciones automaticamente en respuesta a eventos              | Subida de foto a S3 dispara la funcion de clasificacion IA           |

#### 4. Aplicacion de patrones de disenno

| Patron de disenno | Que problema resuelve?                                                     | Donde se aplica?                                                                          |
|-------------------|----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Factory           | Crear el procesador de imagen adecuado segun el tipo de analisis requerido | Funcion de clasificacion: crear el clasificador correcto (paisaje, rostro, comida)        |
| Adapter           | Integrar diferentes proveedores de IA con una interfaz uniforme            | Conectar con AWS Rekognition, Google Vision o Azure Computer Vision sin cambiar la logica |
| Builder           | Construir la respuesta del album paso a paso con multiples atributos       | Generacion del album: titulo, fotos, categoria, fecha, permisos de acceso                 |

#### 5. Diseno basico de la arquitectura



#### Por que esta combinacion es adecuada?

- **Costo eficiente:** Solo se paga por cada foto procesada; sin costo cuando no hay actividad.
- **Escalabilidad automatica:** Si 1000 usuarios suben fotos simultaneamente, las funciones se escalan automaticamente.
- **Simplicidad operativa:** No hay servidores que administrar, parchear o monitorear.
- **Extensibilidad:** El patron Adapter permite cambiar de proveedor de IA sin modificar la logica de las funciones.



## Tabla Resumen Comparativa

La siguiente tabla resume los cinco ejemplos para facilitar la comparacion entre estilos:

| Estilo Arquitectonico | Aplicacion de Ejemplo                          | Patrones Arquitectonicos                                                                    | Patrones de Diseno                |
|-----------------------|------------------------------------------------|---------------------------------------------------------------------------------------------|-----------------------------------|
| Microservicios        | MiReservaPro<br>(Reservas online)              | API Gateway, Service Discovery, Circuit Breaker, Database per Service, Event-Driven Pub/Sub | Factory, Singleton, Observer      |
| Cliente-Servidor      | GestorDocumental<br>(Gestion de archivos)      | MVC, Repository, DTO                                                                        | Singleton, Strategy, Adapter      |
| Arquitectura en Capas | AcademiaOnline<br>(Plataforma educativa)       | MVC, Repository, Service Layer                                                              | Factory Method, Observer, Builder |
| Event-Driven          | LogiTrack<br>(Logistica en tiempo real)        | Event Bus, CQRS, Event Sourcing                                                             | Observer, Strategy, Singleton     |
| Serverless            | FotoAlbumIA<br>(Clasificacion de fotos con IA) | API Gateway, BFF, Event-Driven Triggers                                                     | Factory, Adapter, Builder         |

## Instrucciones para el Taller del Estudiante

Con base en los cinco ejemplos presentados en esta guía, el estudiante debe realizar la siguiente actividad practica:

### Actividad: Diseñar una aplicacion web

#### Requisitos de entrega:

- Elegir un estilo arquitectonico de los cinco estudiados y justificar la eleccion.
- Aplicar al menos 2 patrones arquitectonicos, indicando que problema resuelven y donde se aplican.
- Usar al menos 3 patrones de diseno en el codigo, explicando su funcion dentro de la aplicacion.
- Incluir un diagrama basico de la arquitectura propuesta.
- Justificar por que la combinacion de estilo y patrones es adecuada para la aplicacion elegida.

### Formato de entrega

El trabajo debe seguir la misma estructura de los ejemplos de esta guía:

- Seccion 1: Descripcion del caso de uso (nombre, objetivo, funciones clave)
- Seccion 2: Selecccion y justificacion del estilo arquitectonico
- Seccion 3: Tabla de patrones arquitectonicos aplicados
- Seccion 4: Tabla de patrones de diseno aplicados
- Seccion 5: Diseno basico de la arquitectura (diagrama)
- Seccion 6: Justificacion de por que la combinacion es adecuada

### Criterios de evaluacion

| Criterio                           | Descripcion                                                            | Peso |
|------------------------------------|------------------------------------------------------------------------|------|
| Correcta identificacion del estilo | El estilo seleccionado es coherente con el tipo de aplicacion          | 20%  |
| Patrones arquitectonicos           | Se aplican al menos 2 patrones y se justifica su uso                   | 25%  |
| Patrones de diseno                 | Se aplican al menos 3 patrones y se explica donde se usan en el codigo | 25%  |
| Diagrama de arquitectura           | El diagrama refleja correctamente la estructura propuesta              | 15%  |
| Justificacion y coherencia         | La combinacion de estilo y patrones es coherente y bien argumentada    | 15%  |